Algorithm Complexity

Tutorial 2

Exercise 1

Consider an algorithm Selection Sort to sort n numbers stored in a table A. Selection Sort starts by finding the smallest element in A and swapping it with the first element of A. Then, the algorithm finds the second smallest element and swaps it with the second position. It continues this process for the remaining elements.

1. Write the algorithm in pseudo-code.
2. State the loop invariant of the algorithm and prove its correctness.
3. Find the best-case and worst-case time complexity.
4. Calculate the algorithm's complexity based on fundamental operations.

Solution

1. **Pseudocode for Selection-Sort**

   The algorithm repeatedly finds the smallest element in the unsorted part of the table and places it at the start in this part.

   Algorithm 1: Selection-Sort(A)

```
for i ← 1 to length(A) − 1 do
    m ← i;
    for j ← i + 1 to length(A) do
        if A[j] < A[m] then
            m ← j;
        end
    end
    if m ≠ i then
        temp ← A[i]; A[i] ← A[m]; A[m] ← temp;
    end
end
```

   The condition $m \neq i$ avoids an unnecessary swap when the smallest element is already in position i.

2. **Loop invariant and correctness demonstration**

   At the start of each iteration of the outer loop, the following invariant holds:

   $$\boxed{A[1..i-1] \text{ contains the } i-1 \text{ smallest elements of the original table in sorted order.}}$$

   Initialization:

   Before the first iteration,

   $$i = 1.$$

   The subtable $A[1..0]$ is empty, so it trivially contains the 0 smallest elements in sorted order. Therefore, the invariant holds.

   Maintenance:

   Assume that the invariant holds at the start of an iteration with index i.

   The inner loop searches the subtable

   $$A[i..n]$$

   and identifies the position m of its smallest element.

   This element is then placed at position i. Since all elements in

$$A[1..i-1]$$

are already the $i-1$ smallest elements in sorted order, placing the smallest element from $A[i..n]$ at position i gives

$$A[1..i]$$

containing the i smallest elements in sorted order.

Thus, the invariant remains valid for the next iteration.

Termination:

The outer loop terminates after the iteration $i = n-1$.

At this point,

$$A[1..n-1]$$

contains the $n-1$ smallest elements in sorted order. The remaining element $A[n]$ must therefore be the largest element.

Hence,

$$A[1] \leq A[2] \leq \cdots \leq A[n],$$

and the table is sorted.

SELECTION-SORT correctly sorts the input table.

3. **Best-case and worst-case time complexity**

The fundamental operation is the comparison

$$A[j] < A[m].$$

For every value of i, the inner loop performs exactly

$$n - i$$

comparisons.

Therefore, the total number of comparisons is

$$C(n) = \sum_{i=1}^{n-1} (n-i).$$

Using

$$\sum_{i=1}^{n-1} i = \frac{n(n-1)}{2},$$

we obtain

$$C(n) = \frac{n(n-1)}{2}.$$

Thus,

$$\boxed{C(n) = \frac{n(n-1)}{2} = \Theta(n^2)}.$$

This number is exactly the same regardless of the initial ordering of the table.

Consequently,

$$\boxed{T_{\text{best}}(n) = \Theta(n^2)}$$

and

$$\boxed{T_{\text{worst}}(n) = \Theta(n^2)}.$$

In fact, the average case also has the same asymptotic complexity:

$$\boxed{T_{\text{average}}(n) = \Theta(n^2)}.$$

The reason is that Selection Sort always scans the complete unsorted part of the table to identify its smallest element.

4. **Complexity based on fundamental operations**

Let $C(n)$ denote the number of executions of the fundamental operation.

We have

$$C(n) = (n-1) + (n-2) + \cdots + 2 + 1.$$

Therefore,

$$C(n) = \frac{n(n-1)}{2} = \frac{n^2 - n}{2}.$$

Hence,

$$\boxed{C(n) = \Theta(n^2)}.$$

The number of assignments and swaps is smaller:

$$\text{at most } n-1 \text{ swaps.}$$

Thus,

$$\text{number of swaps} = O(n),$$

while

$$\text{number of comparisons} = \Theta(n^2).$$

Since the quadratic number of comparisons dominates the linear number of swaps and other constant-time operations, the overall running time is

$$\boxed{\Theta(n^2)}.$$

The complete complexity summary is therefore:

Case	Time Complexity
Best case	$\Theta(n^2)$
Average case	$\Theta(n^2)$
Worst case	$\Theta(n^2)$
Additional space	$\Theta(1)$

Important observation

Unlike algorithms such as Insertion-Sort, Selection Sort does not significantly take advantage from an already sorted table. Even when the table is already sorted, it must still perform

$$\frac{n(n-1)}{2}$$

comparisons.

Therefore, the best, average, and worst cases all have the same asymptotic order:

$$\boxed{\Theta(n^2)}.$$

Exercise 2

Consider the Quick Sort algorithm for sorting n numbers stored in a table A. The algorithm selects an element called the *pivot*, partitions the table so that elements smaller than the pivot are placed before it and elements greater than the pivot are placed after it, and then recursively sorts the two resulting subtables.

1. Write the pseudocode for the Quick Sort algorithm and its Partition procedure.
2. For the table
$$A = \langle 7, 3, 7, 4, 9, 2, 6, 1, 7, 5 \rangle,$$
trace the first partitioning step using the last element as the pivot. Show the resulting table and the final position of the pivot.
3. Establish the recurrence relation for the running time of Quick Sort in:
   (a) the best case, when the pivot divides the table into two approximately equal parts;
   (b) the worst case, when the pivot produces one subtable containing $n-1$ elements and another containing no elements.
4. Determine the asymptotic time complexity of Quick Sort in the best and worst cases. Explain the reason for the difference between these two cases.
5. State the loop invariant of the main loop in the Partition procedure and use it to justify that, when the procedure terminates, the pivot is placed in its correct position.

Solution

1. **Pseudocode of Quick Sort and Partition**

   The Quick Sort algorithm follows the divide-and-conquer strategy. The table is partitioned around a pivot, and the two resulting subtables are sorted recursively.

```
1 if p < r then
2 |   pivot ← Partition(A, p, r);
3 |   Quick-Sort(A, p, pivot − 1);
4 |   Quick-Sort(A, pivot + 1, r);
5 end
```

Algorithm 2: Quick-Sort(A, p, r)

```
1 x ← A[r]; i ← p − 1;
2 for j ← p to r − 1 do
3 |   if A[j] ≤ x then
4 |   |   i ← i + 1;
5 |   |   permute A[i] and A[j];
6 |   end
7 end
8 permute A[i + 1] and A[r];
9 return i + 1;
```

Algorithm 3: Partition(A, p, r)

   The Partition procedure uses the last element as the pivot.

   The procedure places the pivot between the elements smaller than or equal to it and the elements greater than it.

2. **First partitioning step**

Consider

$$A = \langle 9, 3, 7, 4, 9, 2, 6, 1, 7, 5 \rangle.$$

The last element is selected as the pivot:

$$x = 5.$$

Initially,

$$i = 0.$$

We scan the elements from left to right.

j	$A[j]$	Comparison with 5	Action
1	7	$7 > 5$	No permutation
2	3	$3 \leq 5$	$i \leftarrow 1; A[1] \leftrightarrow A[2]$
3	7	$7 > 5$	No permutation
4	4	$4 \leq 5$	$i \leftarrow 2; A[2] \leftrightarrow A[4]$
5	9	$9 > 5$	No permutation
6	2	$2 \leq 5$	$i \leftarrow 3; A[3] \leftrightarrow A[6]$
7	6	$6 > 5$	No permutation
8	1	$1 \leq 5$	$i \leftarrow 4; A[4] \leftrightarrow A[8]$
9	7	$7 > 5$	No permutation

After processing all elements before the pivot, the table becomes

$$\langle 3, 4, 2, 1, 9, 7, 6, 7, 7, 5 \rangle.$$

Finally, the pivot 5 is permuted with $A[i+1] = A[4]$:

$$\langle 3, 4, 2, 1, 5, 7, 6, 7, 7, 9 \rangle.$$

Therefore, the pivot is placed at position

$$\boxed{q = 5}.$$

The resulting subtables are

$$\langle 3, 4, 2, 1 \rangle \qquad \text{and} \qquad \langle 7, 6, 7, 7, 9 \rangle.$$

Hence,

$$\boxed{\langle 3, 4, 2, 1, \mathbf{5}, 7, 6, 7, 7, 9 \rangle}$$

is the result of the first partitioning step.

3. **Recurrence relations**

The partition procedure examines each element of the current subtable once. Therefore, partitioning n elements takes

$$\Theta(n)$$

time.

Best case:

In the best case, the pivot divides the table into two approximately equal parts:

$$T(n) = 2T\left(\frac{n}{2}\right) + cn,$$

where cn represents the cost of partitioning.

Using the Master Theorem,

$$T(n) = \Theta(n \log n).$$

Thus,

$$\boxed{T_{\text{best}}(n) = \Theta(n \log n)}.$$

Worst case:

In the worst case, the pivot is always the smallest or largest element. One subtable contains $n-1$ elements and the other contains no elements:

$$T(n) = T(n-1) + cn.$$

Expanding the recurrence:

$$T(n) = T(n-1) + cn \quad = T(n-2) + c(n-1) + cn \ = \cdots \quad = T(1) + c(2+3+4+\cdots+n).$$

Since

$$2 + 3 + 4 + \cdots + n = \frac{n(n+1)}{2} - 1,$$

we obtain

$$T(n) = \Theta(n^2).$$

Therefore,

$$\boxed{T_{\text{worst}}(n) = \Theta(n^2)}.$$

4. **Best-case and worst-case complexities**

The results can be summarized as follows:

Case	Time Complexity
Best case	$\Theta(n \log n)$
Worst case	$\Theta(n^2)$

The difference comes from the quality of the pivot.

When the pivot divides the table into two approximately equal parts, the recursion depth is approximately

$$\log_2 n,$$

and each level performs a total of $\Theta(n)$ partitioning work. Therefore,

$$\Theta(n) \times \Theta(\log n) = \Theta(n \log n).$$

In the worst case, the recursion becomes highly unbalanced:

$$n, n-1, n-2, \ldots, 1.$$

The total work is therefore

$$n + (n-1) + (n-2) + \cdots + 1 = \Theta(n^2).$$

5. **Loop invariant of Partition**

At the start of each iteration of the main loop, with pivot $x = A[r]$, the following invariant holds:

$$\boxed{\begin{aligned} A[p..i] &\leq x, \\ A[i+1..j-1] &> x. \end{aligned}}$$

In other words:

* all elements in $A[p..i]$ are less than or equal to the pivot; * all elements in $A[i+1..j-1]$ are greater than the pivot; * the elements from $A[j]$ onward have not yet been processed.

Initialization:

Before the first iteration,

$$i = p - 1, \qquad j = p.$$

Thus, both processed regions are empty, so the invariant is true.

Maintenance:

If

$$A[j] \leq x,$$

then i is increased and $A[j]$ is permuted with $A[i]$. Consequently, the element added to $A[p..i]$ is less than or equal to the pivot.

If

$$A[j] > x,$$

no permutation is performed. The element remains in the region containing elements greater than the pivot.

Therefore, the invariant remains true after each iteration.

Termination:

When the loop terminates,

$$j = r.$$

Therefore, every element in $A[p..r-1]$ has been processed. The invariant gives

$$A[p..i] \leq x \text{ and } A[i+1..r-1] > x.$$

The final permutation places the pivot at position $i + 1$. Hence,

$$A[p..i] \leq A[i+1] \text{ and } A[i+2..r] > A[i+1].$$

Thus, the pivot is placed in its correct position, and the PARTITION procedure is correct.

Conclusion

QUICK SORT illustrates an important principle in algorithm analysis: the running time depends not just on the algorithm itself but also on how the input is divided during execution. A balanced partition leads to

$$\Theta(n \log n),$$

whereas highly unbalanced partitions can lead to

$$\Theta(n^2).$$

The choice of pivot is therefore an important factor in the practical and theoretical performance of QUICK SORT.